

DATA STRUCTURES & ALGORITHMS in Python

12 Essential Problems • Crystal-Clear Explanations • Production-Quality Code

The only short collection you need to build real confidence for interviews

■ Fast Learning Clear approach + why it works	■ Deep Insight Key patterns & edge cases	■ Clean Code Type-hinted & tested
■ Real Complexity Time & Space analysis	■ Practice Links Solve on LeetCode instantly	■ GitHub Ready Upload & showcase today

Topics you will master

Arrays • Hash Maps • Stacks • Linked Lists • Trees • Dynamic Programming • Sliding Window patterns • Greedy thinking

Every problem is chosen because it teaches a reusable pattern that appears again and again in real interviews.

Turn the page → Start solving smarter

Quick Navigation

1	Two Sum	Hash Map	Easy
2	Best Time to Buy and Sell Stock	One Pass	Easy
3	Contains Duplicate	Hash Set	Easy
4	Product of Array Except Self	Prefix / Suffix	Medium
5	Maximum Subarray	Kadane's Algo	Medium
6	Merge Intervals	Sorting + Merge	Medium
7	Valid Parentheses	Stack	Easy
8	Reverse Linked List	Pointers	Easy
9	Merge Two Sorted Lists	Dummy Node	Easy
10	Binary Tree Inorder Traversal	Tree / Stack	Easy
11	Climbing Stairs	DP / Fibonacci	Easy
12	Coin Change	Unbounded DP	Medium

How to get maximum value from this PDF

1. Read the problem → try solving it yourself for 10–15 minutes.
2. Only then look at the approach and code.
3. Click the **Solve on LeetCode** link and submit your own solution.
4. Come back and compare your code with the clean version here.

■ **Pro tip:** After finishing all 12, try to re-solve any 3 problems from scratch the next day. That is how patterns stick.

1. Two Sum

Topic: Arrays / Hash Map | **Difficulty:** Easy

Problem

Given an array of integers **nums** and an integer **target**, return the indices of the two numbers such that they add up to target. Exactly one solution exists. You may not use the same element twice.

Examples

- `nums = [2,7,11,15]`, `target = 9` → `[0,1]`
- `nums = [3,2,4]`, `target = 6` → `[1,2]`

Approach — How the pros think

One-pass Hash Map is the elegant solution. While you iterate, store each number and its index. For the current number `x`, ask: “Have I already seen `target - x`?” If yes, you found the pair instantly. This is the classic trade-off of space for speed.

■ Key Insight

Most ‘find pair’ problems can be solved with a Hash Map in $O(n)$ instead of $O(n^2)$. Train your brain to ask: “What am I looking for? Can I store the complement?”

Complexity: Time $O(n)$ • Space $O(n)$

Python Solution

```
from typing import List

def two_sum(nums: List[int], target: int) -> List[int]:
    seen = {} # value → index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

■ [Solve “Two Sum” on LeetCode right now →](#)

2. Best Time to Buy and Sell Stock

Topic: Arrays | **Difficulty:** Easy

Problem

You are given an array **prices** where `prices[i]` is the price of a stock on day *i*. Choose one day to buy and a later day to sell. Return the maximum profit. If no profit is possible, return 0.

Examples

- `prices = [7,1,5,3,6,4]` → 5 (buy at 1, sell at 6)
- `prices = [7,6,4,3,1]` → 0

Approach — How the pros think

Keep a running minimum price. Every day, calculate what profit you would make if you sold today, and keep the best profit seen so far. One beautiful pass — no need to look into the future explicitly.

■ Key Insight

Many 'max profit / max difference' problems reduce to tracking a running minimum (or maximum) while scanning left to right.

Complexity: Time $O(n)$ • Space $O(1)$

Python Solution

```
from typing import List

def max_profit(prices: List[int]) -> int:
    min_price = float('inf')
    best = 0
    for price in prices:
        if price < min_price:
            min_price = price
        else:
            best = max(best, price - min_price)
    return best
```

■ [Solve “Best Time to Buy and Sell Stock” on LeetCode right now →](#)

3. Contains Duplicate

Topic: Arrays / Hash Set | **Difficulty:** Easy

Problem

Given an integer array **nums**, return true if any value appears at least twice, otherwise return false.

Examples

- `nums = [1,2,3,1] → true`
- `nums = [1,2,3,4] → false`

Approach — How the pros think

The simplest correct solution uses a Hash Set. Insert elements one by one. If an element is already present → duplicate found → return True immediately. Early exit makes it fast in practice.

■ Key Insight

Whenever you need to answer “have I seen this before?”, reach for a Hash Set first. Sorting is a valid alternative but slower ($O(n \log n)$).

Complexity: Time $O(n)$ average • Space $O(n)$

Python Solution

```
from typing import List

def contains_duplicate(nums: List[int]) -> bool:
    seen = set()
    for num in nums:
        if num in seen:
            return True
        seen.add(num)
    return False
```

■ [Solve “Contains Duplicate” on LeetCode right now →](#)

4. Product of Array Except Self

Topic: Arrays | **Difficulty:** Medium

Problem

Return an array answer such that $\text{answer}[i]$ equals the product of all elements of `nums` except `nums[i]`. Solve it in $O(n)$ time and without using division.

Examples

- `nums = [1, 2, 3, 4] → [24, 12, 8, 6]`
- `nums = [-1, 1, 0, -3, 3] → [0, 0, 9, 0, 0]`

Approach — How the pros think

Two passes. First left-to-right: store the product of everything before the current index (prefix). Then right-to-left: multiply by the product of everything after (suffix). The result at each position becomes $\text{prefix} \times \text{suffix}$.

■ Key Insight

When division is forbidden or zeros appear, prefix/suffix products are the standard technique. Extremely common in interview follow-ups.

Complexity: Time $O(n)$ • Space $O(1)$ extra

Python Solution

```
from typing import List

def product_except_self(nums: List[int]) -> List[int]:
    n = len(nums)
    result = [1] * n
    prefix = 1
    for i in range(n):
        result[i] = prefix
        prefix *= nums[i]
    suffix = 1
    for i in range(n-1, -1, -1):
        result[i] *= suffix
        suffix *= nums[i]
    return result
```

■ [Solve “Product of Array Except Self” on LeetCode right now →](#)

5. Maximum Subarray

Topic: Arrays • Kadane | **Difficulty:** Medium

Problem

Find the contiguous subarray (containing at least one number) which has the largest sum and return its sum.

Examples

- `nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4] → 6 → [4, -1, 2, 1]`
- `nums = [5, 4, -1, 7, 8] → 23`

Approach — How the pros think

Kadane's Algorithm is pure elegance. At every step decide: "Is it better to extend the previous subarray or start a brand-new one from here?" Keep the global maximum of those local decisions.

■ Key Insight

This is the prototype of many DP-on-arrays problems. Once you internalize the 'extend or restart' decision, a whole family of problems becomes easy.

Complexity: Time $O(n)$ • Space $O(1)$

Python Solution

```
from typing import List

def max_sub_array(nums: List[int]) -> int:
    max_current = max_global = nums[0]
    for num in nums[1:]:
        max_current = max(num, max_current + num)
        max_global = max(max_global, max_current)
    return max_global
```

■ [Solve "Maximum Subarray" on LeetCode right now →](#)

6. Merge Intervals

Topic: Sorting + Linear Scan | **Difficulty:** Medium

Problem

Given an array of intervals [start, end], merge all overlapping intervals and return the non-overlapping intervals that cover the input.

Examples

- `[[1, 3], [2, 6], [8, 10], [15, 18]] → [[1, 6], [8, 10], [15, 18]]`
- `[[1, 4], [4, 5]] → [[1, 5]]`

Approach — How the pros think

Sort by start time first. Then walk once: if the current interval overlaps the last merged one, extend the end; otherwise push a new interval. Sorting turns a hard problem into a simple linear pass.

■ Key Insight

Almost every interval problem starts with “sort by start (or end)”. After that the solution is usually a greedy linear scan.

Complexity: Time $O(n \log n)$ • Space $O(n)$

Python Solution

```
from typing import List

def merge(intervals: List[List[int]]) -> List[List[int]]:
    if not intervals:
        return []
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for curr in intervals[1:]:
        last = merged[-1]
        if curr[0] <= last[1]:
            last[1] = max(last[1], curr[1])
        else:
            merged.append(curr)
    return merged
```

■ [Solve “Merge Intervals” on LeetCode right now →](#)

7. Valid Parentheses

Topic: Stack | **Difficulty:** Easy

Problem

Given a string containing only `()[]{}` , decide whether the brackets are valid: every open bracket is closed by the same type and in the correct order.

Examples

- `s = "()[]{}"` → `true`
- `s = "([)]"` → `false`
- `s = "{[]}"` → `true`

Approach — How the pros think

Classic stack problem. Push opening brackets. When you see a closing bracket, the top of the stack must be its matching open bracket. At the end the stack must be empty. A simple map makes matching trivial.

■ Key Insight

Stacks are the natural tool whenever the problem talks about nesting, matching, or 'last opened must be closed first'.

Complexity: Time $O(n)$ • Space $O(n)$

Python Solution

```
def is_valid(s: str) -> bool:
    stack = []
    mapping = {'(': ')', '[': ']', '{': '}'}
    for char in s:
        if char in mapping:
            # closing
            top = stack.pop() if stack else '#'
            if mapping[char] != top:
                return False
        else:
            stack.append(char)
    return not stack
```

■ [Solve “Valid Parentheses” on LeetCode right now →](#)

8. Reverse Linked List

Topic: Linked List | **Difficulty:** Easy

Problem

Given the head of a singly linked list, reverse the list and return the new head.

Examples

- [1, 2, 3, 4, 5] → [5, 4, 3, 2, 1]
- [1, 2] → [2, 1]

Approach — How the pros think

Three pointers: prev, curr, next. At each step save next, reverse the link of curr, then advance. The iterative version uses constant space and is preferred in interviews. Drawing the pointers on paper for 30 seconds removes all confusion.

■ Key Insight

Mastering pointer manipulation on linked lists is a rite of passage. Once reverse is muscle memory, many harder list problems become approachable.

Complexity: Time $O(n)$ • Space $O(1)$

Python Solution

```
from typing import Optional

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def reverse_list(head: Optional[ListNode]) -> Optional[ListNode]:
    prev, curr = None, head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    return prev
```

■ [Solve “Reverse Linked List” on LeetCode right now →](#)

9. Merge Two Sorted Lists

Topic: Linked List | **Difficulty:** Easy

Problem

Merge two sorted linked lists into one sorted list by splicing the existing nodes together. Return the head of the merged list.

Examples

- $[1, 2, 4] + [1, 3, 4] \rightarrow [1, 1, 2, 3, 4, 4]$
- $[] + [0] \rightarrow [0]$

Approach — How the pros think

Dummy node is the secret weapon. It removes all special cases for the head. Compare the two current nodes, attach the smaller one, and advance. When one list is exhausted, simply attach the remainder of the other.

■ Key Insight

Dummy / sentinel nodes eliminate edge-case headaches in almost every linked-list construction problem. Use them liberally.

Complexity: Time $O(n+m)$ • Space $O(1)$ extra

Python Solution

```
from typing import Optional

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def merge_two_lists(l1, l2):
    dummy = ListNode(0)
    curr = dummy
    while l1 and l2:
        if l1.val <= l2.val:
            curr.next = l1
            l1 = l1.next
        else:
            curr.next = l2
            l2 = l2.next
        curr = curr.next
    curr.next = l1 or l2
    return dummy.next
```

■ [Solve “Merge Two Sorted Lists” on LeetCode right now →](#)

10. Binary Tree Inorder Traversal

Topic: Tree | **Difficulty:** Easy

Problem

Return the inorder traversal (Left → Root → Right) of a binary tree's node values.

Examples

- root = [1,null,2,3] → [1,3,2]
- root = [] → []

Approach — How the pros think

Recursive version is one line of real logic. The iterative version uses an explicit stack to simulate the call stack: go left as far as possible, process, then go right. Both are worth knowing.

■ Key Insight

Inorder of a Binary Search Tree yields sorted order. That single fact is used in countless BST problems.

Complexity: Time **O(n)** • Space **O(h)** (height)

Python Solution

```
from typing import Optional, List

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def inorder_traversal(root: Optional[TreeNode]) -> List[int]:
    result, stack, curr = [], [], root
    while curr or stack:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        result.append(curr.val)
        curr = curr.right
    return result
```

■ [Solve “Binary Tree Inorder Traversal” on LeetCode right now →](#)

11. Climbing Stairs

Topic: Dynamic Programming | **Difficulty:** Easy

Problem

You can climb 1 or 2 steps at a time. In how many distinct ways can you climb a staircase of n steps?

Examples

- $n = 2 \rightarrow 2$
- $n = 3 \rightarrow 3$
- $n = 5 \rightarrow 8$

Approach — How the pros think

Classic Fibonacci in disguise. The number of ways to reach step i is $\text{ways}(i-1) + \text{ways}(i-2)$. You only need the previous two values, so space collapses to $O(1)$.

■ Key Insight

Many counting problems are secret Fibonacci (or simple linear DP). Always ask: "What are the last decisions I could have made?"

Complexity: Time $O(n)$ • Space $O(1)$

Python Solution

```
def climb_stairs(n: int) -> int:
    if n <= 2:
        return n
    prev2, prev1 = 1, 2
    for _ in range(3, n+1):
        curr = prev1 + prev2
        prev2, prev1 = prev1, curr
    return prev1
```

■ [Solve "Climbing Stairs" on LeetCode right now →](#)

12. Coin Change

Topic: Dynamic Programming | **Difficulty:** Medium

Problem

Given coins of different denominations and a total amount, return the fewest number of coins needed to make that amount. Return -1 if it is impossible. You have infinite supply of each coin.

Examples

- coins = [1,2,5], amount = 11 → 3
- coins = [2], amount = 3 → -1

Approach — How the pros think

Bottom-up DP. $\text{dp}[x]$ = minimum coins to make amount x . For every amount, try every coin and take the minimum. This is the classic unbounded-knapsack pattern.

■ Key Insight

When you hear 'minimum number of items with unlimited supply', think of this exact DP. Master the loop order — it matters.

Complexity: Time $O(\text{amount} \times \text{coins})$ • Space $O(\text{amount})$

Python Solution

```
from typing import List

def coin_change(coins: List[int], amount: int) -> int:
    dp = [amount + 1] * (amount + 1)
    dp[0] = 0
    for x in range(1, amount + 1):
        for coin in coins:
            if coin <= x:
                dp[x] = min(dp[x], dp[x - coin] + 1)
    return dp[amount] if dp[amount] <= amount else -1
```

■ [Solve "Coin Change" on LeetCode right now →](#)

You Finished the Collection ■

If you truly solved (or deeply understood) all 12 problems, you already own the most important patterns that appear in coding interviews:

- Hash Map for pair / complement problems
- Running minimum / maximum for profit & range problems
- Prefix & Suffix techniques
- Kadane's 'extend or restart' thinking
- Sorting + linear scan for intervals
- Stack for matching & nesting
- Pointer manipulation on linked lists
- Tree traversals (recursive + iterative)
- Bottom-up Dynamic Programming (Fibonacci & unbounded knapsack)

Next level challenges

- Re-implement every solution from memory tomorrow.
- Solve the Medium versions / follow-ups of these problems on LeetCode.
- Explain one problem out loud as if you are in a real interview.

This PDF + the accompanying GitHub repository are designed to be your clean, professional starting point. Upload the repo, star it, and keep adding more problems as you grow.

Now go solve. The best way to learn algorithms is to write them.

Happy Coding • Keep the momentum